

APRS Pocket

Anthony Le Cren F4GOH - KF4GOH

1 Introduction

En tant que concepteur de projets électroniques, s'il y a bien un radioamateur que je suis attentivement et depuis des années, c'est bien Burkhard Kainka DK7JD. Ses publications, que ce soit sur son site (1) ou dans la revue Elektor, ont été toujours de bonnes inspirations pour mes projets.

Et justement, dès la lecture d'un de ses articles sur un oscillateur programmable DDS basses fréquences à partir d'un ATTINY1614, j'ai tout de suite pensé à fabriquer une extension tracker APRS pour les TRX pocket FM bon marché.

Certes, le projet n'est pas nouveau : une carte électronique munie d'un microcontrôleur et d'un GPS qui génère le protocole ax25/APRS, le tout relié à un émetteur FM sur la fréquence 144,800 MHz. Mais l'intérêt de l'ATTINY1614 est son prix (1 € 50,) et il intègre directement un convertisseur numérique-analogique (DAC). Attention, le code informatique ne gère que l'émission.

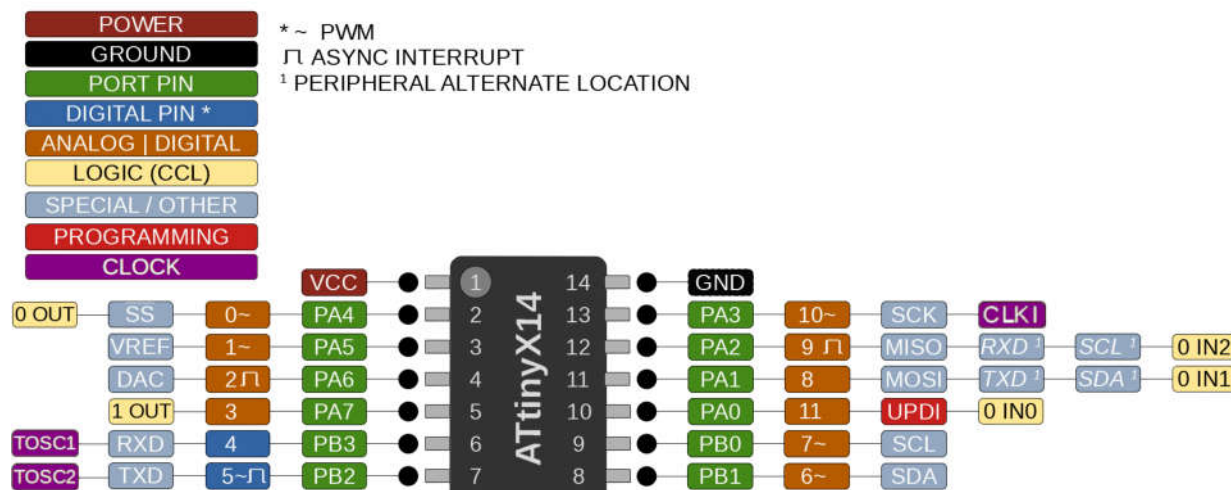
Le décodage n'est pas supporté.

Caractéristiques matérielles de la carte :

- ▶ ATTINY1614 ;
- ▶ GPS VK2828 ;
- ▶ Chargeur TP4056 ;
- ▶ Batterie Lipo 3,7 V.



Le montage relié au pocket.



Les broches de l'ATTINY1614.



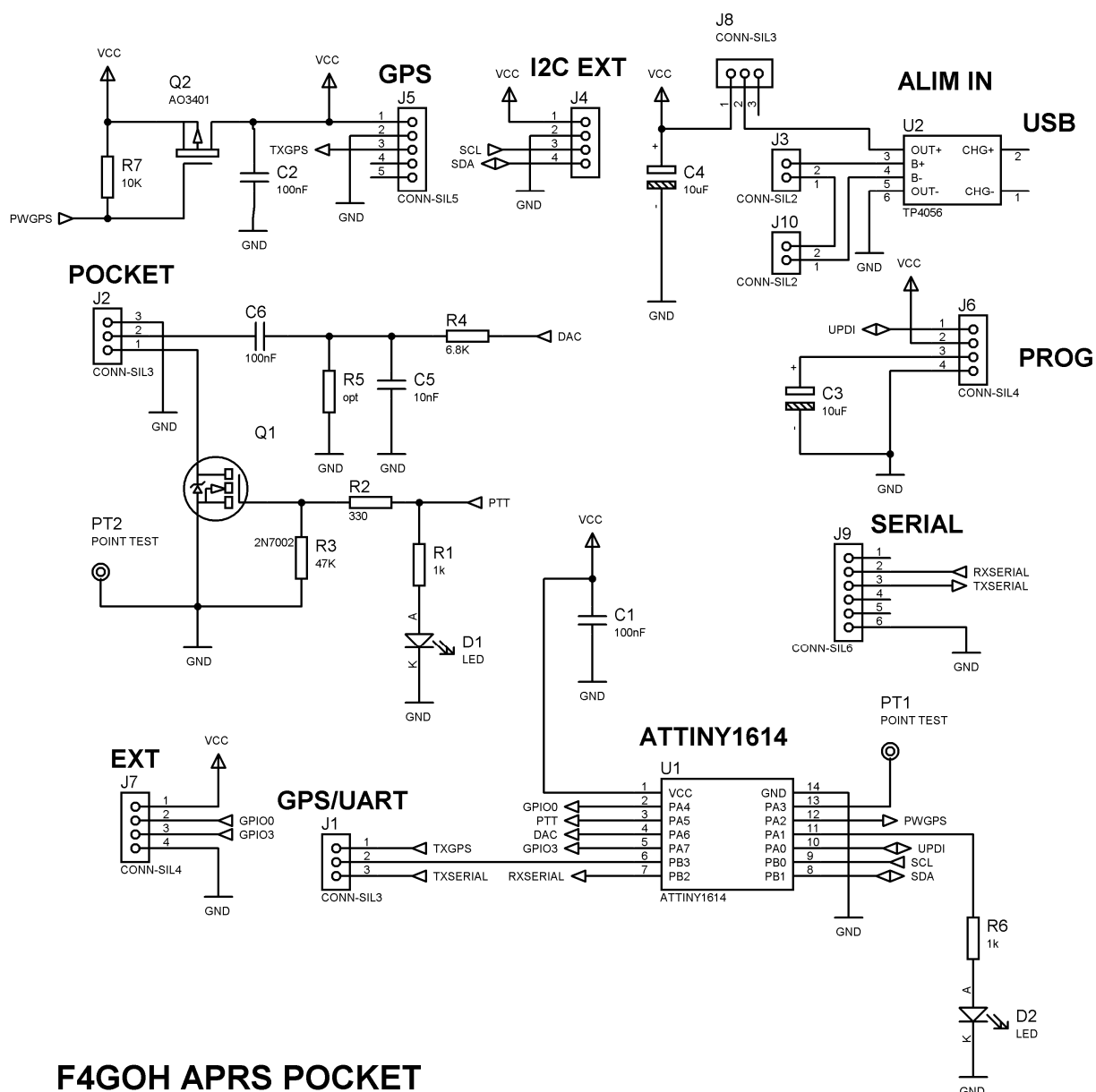
PIN'S DU CENTENAIRE DU REF

PEF013

6,00€

Port non compris

2 Description du schéma structurel



F4GOH APRS POCKET

Le schéma structurel de la carte.

L'ensemble est architecturé autour de l'ATTINY1614. On retrouve la sortie BF du DAC sur la broche PA6. Le signal est filtré par un simple passe-bas du 1^{er} ordre avant d'être injecté dans la prise « micro » du pocket. La commande PTT est réalisée par un transistor Mos 2N7002.

Il n'y a qu'une entrée série UART. Le cavalier J1 permettra de sélectionner la source entre le mode programmation (pour enregistrer son indicatif J9) et le mode utilisation avec le GPS (J5).

Le connecteur J6 est nécessaire pour programmer le logiciel en mode UPDI. J'ai ajouté un chargeur TP4056 pour la batterie Lipo 3,7 V. Il n'y a aucun régulateur de tension car le circuit intégré principal fonctionne très bien entre 3,3 V et 5 V.

Il y a également des broches disponibles pour les personnes désirant modifier mon logiciel. (connecteur J7 et Bus I2C). Mais attention ! L'ATTINY1614 ne dispose que de 16 kb de mémoire Flash.



La carte dans son boîtier. La batterie est sous le PCB et maintenue avec un collier.

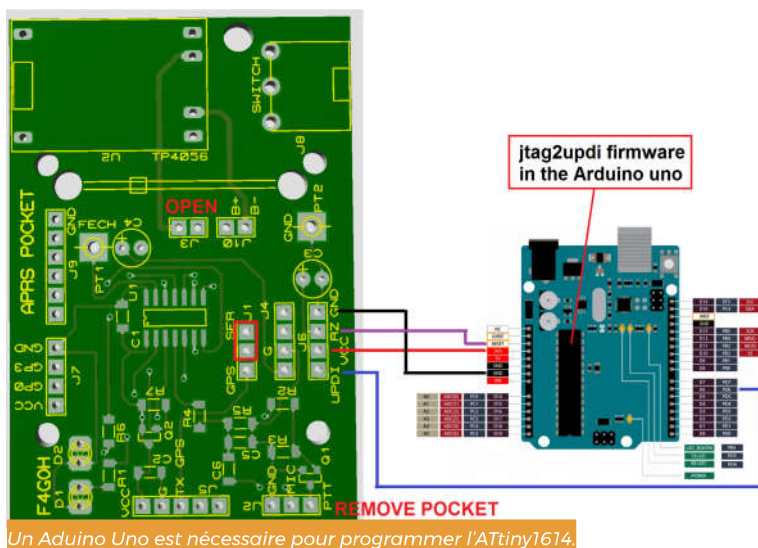
3 Programmation du logiciel

Il faut tout d'abord installer l'environnement de développement « megatinycore ». Pour cela j'ai mis de nombreuses copies d'écran sur mon github(2) pour configurer l'IDE Arduino. Mais on retrouve également de nombreux tutoriels d'installation, comme sur hackster.io (3).



le « framework » spécifique à l'ATtiny1614.

La programmation UPDI signifie Unified Program and Debug Interface. Cette technique de programmation de l'ATtiny1614 est réalisée via une seule broche (PA0), mais il faudra un Arduino Uno qui sera utilisé en tant que programmeur de composant UPDI avec le logiciel jtag2updi préalablement programmé.



Un Arduino Uno est nécessaire pour programmer l'ATtiny1614.

Ensuite vient la sélection des paramètres de l'ATtiny1614 ans l'IDE Arduino :

- ▶ Type de carte , megatinycore, ATtiny1614.
- ▶ Chip , ATtiny1614.
- ▶ Millis()/micros() Timer, disabled.
- ▶ Printf , default.
- ▶ Sélection du port com de l'Arduino UNO.
- ▶ Programmeur « jtag2updi ».

La compilation et le téléversement du programme « APRS-pocket-parser.ino » doivent se faire sans problème. Il n'y a aucune bibliothèque à installer. Le niveau du signal BF est déjà réglé par la référence de tension du convertisseur numérique analogique défini à 1,1 V logiciellement dans le fichier Dds.cpp. La fréquence d'échantillonnage est de 26400 Hz. Cette fréquence n'est pas anodine car $26400/1200=22$ et $26400/2200=12$. Le résultat des deux divisions donne des nombres entiers, ce qui est plutôt pratique pour la compatibilité ax25. Cependant l'oscillateur interne de 20 MHz de l'ATtiny1614 est géré par une cellule RC.

Celle ci n'est pas précise. Même si le programme fonctionne pour la plupart des microcontrôleurs que j'ai testés, il est possible de régler précisément la fréquence d'échantillonnage en ajustant de plus ou moins 1 à 2 le paramètre PER_VALUE (fichier Dds.h). On doit retrouver une fréquence de $26400/2=13200$ Hz sur la broche PA3.

```
#define PER_VALUE 374 //À ajuster pour obtenir 13200 Hz +/- 5 Hz sur PA3 (10)
```

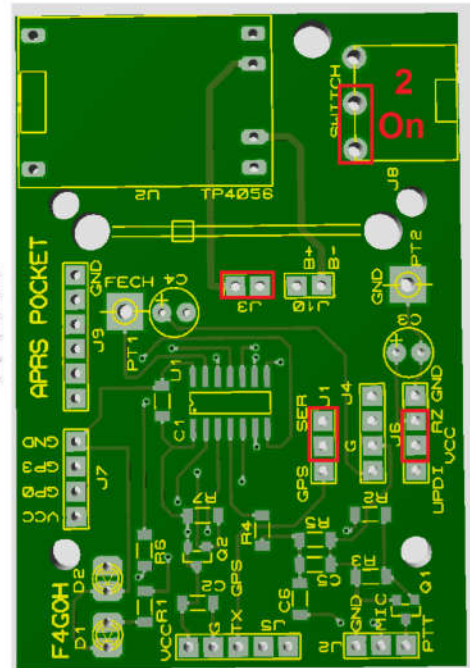
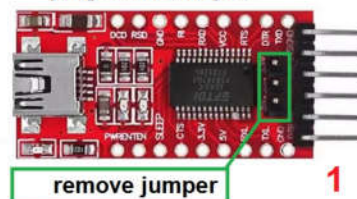
4 Configuration de l'indicatif

La configuration des paramètres de transmission est simplifiée à l'aide d'une interface en ligne (4) sous le navigateur Chrome. En effet, par rapport au navigateur Firefox, le navigateur Chrome peut gérer une liaison série type COMx (Windows ou /dev/ttyUSB0 (Linux).

Par-contre, il faudra utiliser un adaptateur USB série TTL externe pour envoyer la configuration dans l'EEPROM de l'ATtiny1614.

Chrome
Web browser

plug serial adapter



Configuration de l'indicatif et des paramètres de transmission.

La configuration est très simple :

- ▶ L'indicatif et le SSID.
- ▶ Le caractère ASCII correspondant à la table de symbole APRS.
- ▶ Le caractère ASCII du symbole APRS (ici le caractère > correspond à une voiture)
- ▶ L'intervalle de temps en minutes entre deux transmissions..
- ▶ L'intervalle de temps en secondes (si minute =0).
- ▶ La mobilité pour le smartBeaconing (transmission automatique).

- ▶ checkBox smartBeaconing : active la transmission automatique.
- ▶ checkBox Compressed : envoyer des trames compressées.
- ▶ checkBox Altitude : transmet l'altitude dans la trame.
- ▶ checkBox Display : non utilisé pour le moment.

Une fois configuré, il suffit de cliquer sur Connect (connexion) afin de sélectionner le bon port de communication, puis de cliquer sur Send (Envoyer)

